

EP01 — Emergency Department Patient Flow

AI Copilot Blueprint: A Complete Simulation Study

Problem domain:	Healthcare — 30-bed academic emergency department
Simulation tool:	Python 3.11 · SimPy 4.1.1 · NumPy ≥2.0
Frontend:	React 18 · TypeScript 5 · Pyodide 0.27 · Three.js · Vite 6
AI copilot:	claude-sonnet-4-6 (Anthropic · Claude 4 family)
Interface:	Claude Code CLI (VSCode extension)
Workflow stages:	P01 Intake → P02 Conceptual → P03 Scaffold → P04 Debug → P05 Experiments → P06 Analysis → P07 Dashboard
Replications:	50 per scenario · Common Random Numbers · warmup 48 h
Date:	2026-05-24 · Status: Complete

Abstract

This document is an **AI Copilot Blueprint** — a step-by-step record of how a complete discrete-event simulation (DES) study was built using an AI language model as a collaborative engineering partner. It is intended to be read as a workflow guide, not merely a results report. Each section corresponds to one prompt stage: you will see the prompt sent, the key AI output, what was critically reviewed and changed, and the resulting artifact.

The scientific content is a simulation study of a 30-bed academic ED. Baseline *left-without-being-seen* (LWBS) rate is **4.44%** [4.18%, 4.69%], above the 2% target. Three interventions are compared: fast-track with 1 PA (worsens: +0.49 pp), overnight staffing −1 (severe: +3.12 pp), and overnight staffing +1 (best lever: −0.69 pp to 3.75%). Boarding reduction does not move the LWBS rate. The worst patient-experience window is **19:00–21:00**, the physician handover coinciding with peak-arrival residuals. A companion interactive dashboard (Stage 7, Section 7) runs the same Python model in-browser via Pyodide and animates the department in both 2-D and 3-D, demonstrating that AI copilot assistance makes visualization tooling achievable without a dedicated front-end team.

How to read this document: Section 1 explains the AI copilot methodology. Sections 2–7 walk through the seven prompt stages. Sections 8–12 present results and recommendations. Appendices contain verbatim prompts and source code.

Contents

1	AI Copilot Philosophy	2
1.1	The Core Idea	2
1.2	The Six-Stage Workflow	2
1.3	What This Blueprint Is Not	2
2	Stage 1 — Problem Intake (P01)	2
2.1	Objective	2
2.2	The Prompt	3
2.3	AI Response: Entities and Resources	3
2.4	AI Response: Process Sequence	3

3	Stage 2 — Conceptual Model (P02)	3
3.1	Objective	4
3.2	The Prompt	4
3.3	Process Flow Diagram	4
3.4	Conceptual Model Document	5
4	Stages 3 & 4 — Implementation and Debugging (P03, P04)	6
4.1	Stage 3 — The Scaffold (P03)	6
4.1.1	Key patterns the AI got right	6
4.2	Stage 4 — Critical Bug: Physician Held Too Long	7
4.2.1	First run output	7
4.2.2	Diagnosis	8
4.2.3	The fix	8
4.2.4	Calibration	8
4.2.5	Verification checklist	9
5	Stage 5 — Experiment Design (P05)	9
5.1	The Prompt	9
5.2	Replication Count Justification	9
5.3	Common Random Numbers	9
5.4	Scenario Matrix	10
6	Stage 6 — Output Analysis (P06)	10
6.1	The Prompt	10
6.2	Statistical Design	10
7	Stage 7 — Interactive Dashboard (P07)	11
7.1	Motivation: Closing the Visualisation Gap	11
7.2	Architecture	11
7.3	The Prompts	12
7.4	The Dashboard in Detail	12
7.5	What We Accepted and Changed	17
8	Baseline Performance	17
8.1	Patient Volume and Length of Stay	17
8.2	KPI Dashboard	18
9	Door-to-Provider Time by Hour of Arrival	18
9.1	What the Metrics Mean	18
9.2	Hour-by-Hour Interpretation	19
9.2.1	Night 01:00–06:00 — light load, short waits	19
9.2.2	Hour 00:00 (midnight) — counterintuitive high p90	19
9.2.3	Peak 15:00–18:00 — approaching target	19
9.2.4	19:00–21:00 — worst window	19
10	Intervention Scenarios	20
10.1	Scenario 1: Fast-Track (1 PA, 4 Beds)	20
10.2	Scenario 2: Staffing −1 (Night 2 → 1)	21
10.3	Scenario 3: Staffing +1 (Night 2 → 3)	21
11	Boarding Time Sensitivity	21

12 Recommendations	22
13 Limitations	22
14 Conclusion	22
A Verbatim Prompts (P01–P06)	23
A.1 P01 Prompt (excerpt)	23
A.2 P03 Prompt (excerpt)	23
B Key Source Code	23
B.1 Patient Dataclass	23
B.2 Full Patient Process	24
B.3 Reproducibility	25

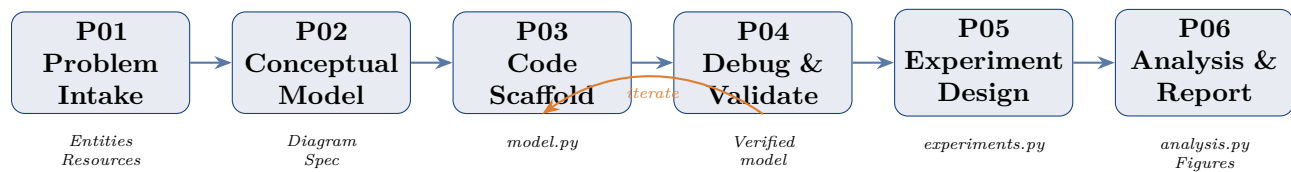
1 AI Copilot Philosophy

1.1 The Core Idea

An AI language model is not an oracle — it is a fast, knowledgeable collaborator that occasionally makes confident mistakes. The productive workflow is:

1. **Write a precise prompt.** Vague prompts produce vague code. Include the system description, exact distributions, and the specific output needed. The prompts in this blueprint took 5–10 minutes each to write; that investment paid off in minutes of AI output.
2. **Read the response critically.** AI code often *runs* but produces wrong answers. Verify against analytical results before trusting complex output. In this study, the first scaffold produced 84% LWBS — plausible-looking code with a conceptually wrong resource model (Section 4.2).
3. **Document every change.** The “What we accepted / changed” note in each section is the most important part. It is where simulation judgment happens. Future readers (and your future self) need to know why the model is the way it is.
4. **Verify quantitatively.** Check the model against analytical bounds, Little’s Law, and limiting cases before running experiments. A model that cannot pass simple sanity checks should not be trusted with scenario comparisons.

1.2 The Six-Stage Workflow



Each stage produces one concrete artifact. The red arrow between P04 and P03 represents the debug–fix loop: the first scaffold required one iteration (physician hold time bug; see Section 4.2).

1.3 What This Blueprint Is Not

This is not a story of AI replacing the engineer. The AI wrote the first draft of every file. The engineer diagnosed the critical bug, made all conceptual corrections, verified all outputs, and designed the experiment structure. The AI accelerated execution; the engineer supplied judgment.

2 Stage 1 — Problem Intake (P01)

2.1 Objective

Before writing a line of SimPy code, we decompose the problem into simulation primitives: entities, resources, events, process logic, and SimPy patterns. Forcing this decomposition first prevents the most common student mistake — jumping into code before understanding the system.

2.2 The Prompt

Prompt sent to claude-sonnet-4-6

You are helping me build a discrete-event simulation of an emergency department using Python and SimPy. Read the problem statement and produce: (1) entity list with attributes, (2) resource list with SimPy type and schedule, (3) event list with trigger and state change, (4) process sequence with all branches, (5) key modeling decisions, (6) SimPy pattern for each resource or delay. Be concrete and use the numerical values from the problem.

[complete EP01 problem statement pasted]

Full verbatim prompt: `prompts/p01_intake.md` (Appendix A)

2.3 AI Response: Entities and Resources

One entity type — **Patient** — with attributes: `pid`, `arrival_time`, `weekend`, `esi` (1–5), timestamps for triage/room/physician start and end, `lab_end`, `imaging_end`, `depart_time`, `disposition` (`discharged` | `admitted` | `lwbs`), `boarding_start/end`.

Resource	SimPy type	Capacity	Schedule
Triage nurses	Resource	2	Constant
Treatment rooms	PriorityResource	30	Constant
Physicians	PriorityResource	varies	4 (07–19), 3 (19–23), 2 (23–07)

Lab and imaging are **pure delays** (`env.timeout`) — not resources. They hold the room but require no contended server.

2.4 AI Response: Process Sequence

arrive

- wait for triage nurse (FCFS)
- triage: `Exp(4 min)`, ESI assigned, nurse released
- wait for treatment room (`priority = ESI`)
- room assigned
- wait for physician (`priority = ESI`) + LWBS 90-min timer starts here
- [LWBS: timer fires → room released → depart]
- physician assigned: assessment hold, then physician released
- lab/imaging ordered (prob by ESI): timeout delay
- disposition: {`discharge` → room released → depart}
- {`admit: boarding Exp(90 min)` → room released → depart}

What we accepted / changed

Accepted: entity attributes, resource table, event list, LWBS `any_of()` pattern, lab/imaging as pure delays.

Modified: AI suggested physician shifts might overlap (“evening 15–23” and “day 07–19”). We modelled sequential handovers at 07:00, 19:00, 23:00 — consistent with the quoted headcounts, not additive shifts. Lab/imaging split fixed at 50/50 (not modelled separately by AI).

Discarded: AI proposed a “fast-track nurse” resource for the baseline. Fast-track is an intervention scenario, not the baseline.

3 Stage 2 — Conceptual Model (P02)

3.1 Objective

Translate the Stage 1 analysis into a visual process-flow diagram and a structured specification document. The diagram is the simulation’s “blueprint” — it must be reviewed and approved before any code is written.

3.2 The Prompt

Prompt sent to claude-sonnet-4-6

I am building a discrete-event simulation of an emergency department. I have already extracted the entity list, resource list, and event list. Now I need: (1) a Mermaid flowchart showing the patient journey from arrival to departure including all branches; (2) a conceptual model document in the standard format (boundary, entities, resources, queues, delays, logic, performance measures). Base the diagram strictly on the analysis below.

[P01 output pasted] *Full prompt:* `prompts/p02_conceptual.md`

3.3 Process Flow Diagram

Figure 1 shows the complete patient journey. The diagram was generated by the AI in Mermaid syntax, then rendered here in TikZ.

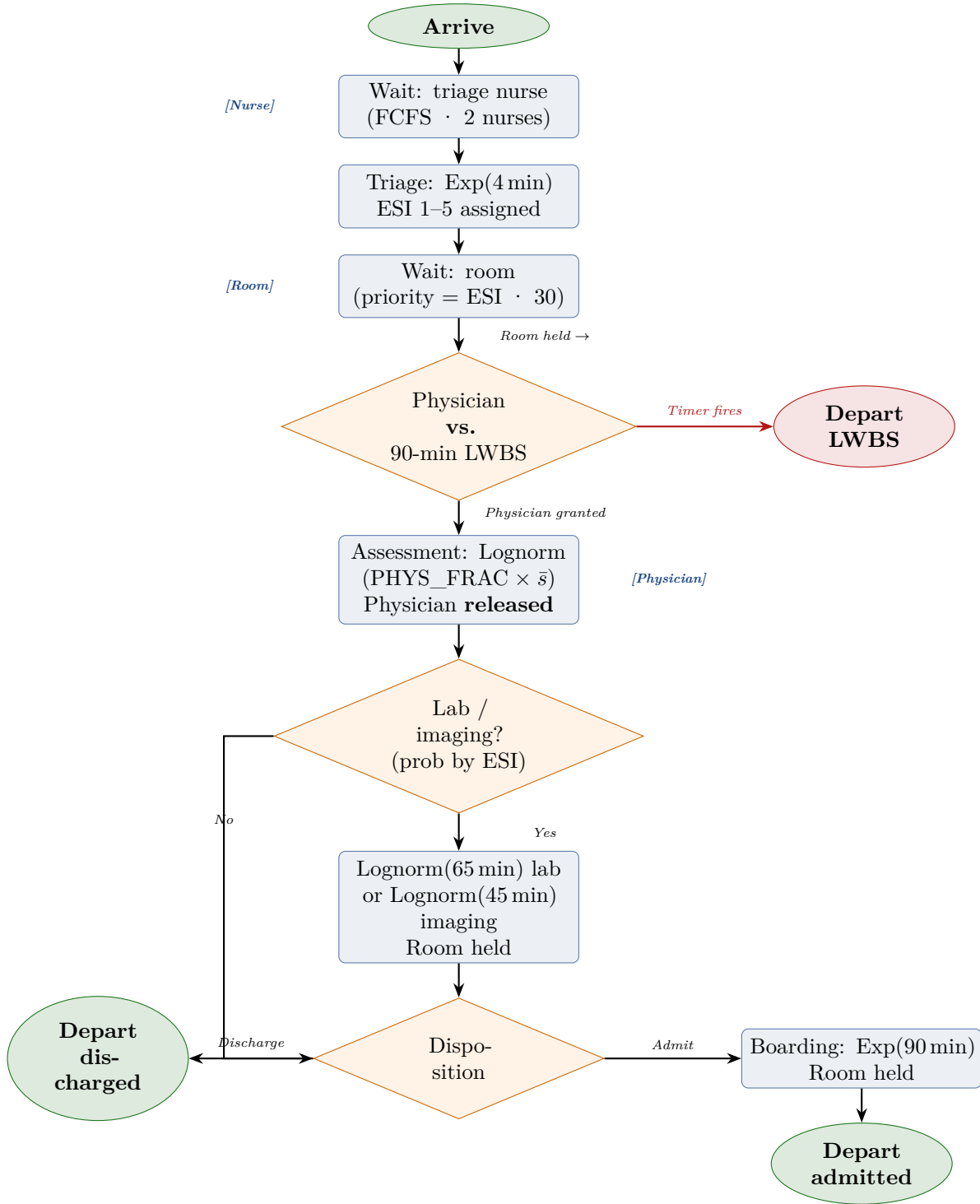


Figure 1: ED patient process flow. Resources seized/released are annotated in blue. The physician is released after the assessment phase; lab/imaging and boarding hold only the room. The LWBS timer runs from room entry.

3.4 Conceptual Model Document

System boundary: *Inside:* triage, treatment rooms, physician encounters, lab/imaging delays, inpatient boarding. *Outside:* ambulance dispatch, inpatient unit dynamics, ICU/OR, psychiatric

boarding, mass-casualty events.

Queue	Resource	Discipline	Patience
Triage queue	Triage nurses	FCFS	None
Room queue	Treatment rooms	Priority (ESI)	None
Physician queue	Physicians	Priority (ESI)	90 min (LWBS)

Delay	Distribution	Mean
Triage time	Exp	4 min
ESI-1 treatment	Lognormal	180 min
ESI-2/3 treatment	Lognormal	135 min
ESI-4/5 treatment	Lognormal	50 min
Lab turnaround	Lognormal	65 min
Imaging turnaround	Lognormal	45 min
Inpatient boarding	Exp	90 min

What we accepted / changed

Two corrections to the AI diagram:

(1) LWBS timer start: AI placed the timer at room assignment. Corrected to *physician queue entry* — the patient is in the room waiting for the provider, and “provider contact” triggers the 90-minute clock per the problem statement.

(2) Boarding room hold: AI initially released the room before boarding. Corrected — the room must be held throughout boarding. This is the whole mechanism by which boarding drives crowding: boarded patients block incoming patients from getting rooms.

4 Stages 3 & 4 — Implementation and Debugging (P03, P04)

Stages 3 and 4 are documented together because they form one iterative loop: scaffold → run → diagnose → fix → verify. Full prompts are in `prompts/p03_scaffold.md` and `prompts/p04_debug.md` (Appendix A).

4.1 Stage 3 — The Scaffold (P03)

The prompt asked for a complete `model.py` matching the conceptual model exactly. The AI produced the following structure on the first generation:

Prompt sent to claude-sonnet-4-6

Write a complete SimPy simulation in Python for the emergency department described in the conceptual model below. Requirements: Python 3.11, SimPy 4.x, NumPy 2.x; *Patient* dataclass; *EDModel* class with *shift_manager()*, *arrival_generator()*, *patient_process()* using nested *with resource.request()* for clean cleanup; *run_replication(seed, warmup, horizon) -> dict*. Use *numpy.random.default_rng(seed)*. No global state.

[conceptual model pasted]

4.1.1 Key patterns the AI got right

Lognormal parametrization. NumPy’s `rng.lognormal( $\mu, \sigma$ )` expects the parameters of the *underlying* normal distribution, not the mean and std of the lognormal. The AI produced the correct helper:

Listing 1: Lognormal parameter conversion (model.py)

```

1 def _lognorm_mu_sigma(mean_min: float, std_min: float) -> tuple[float, float]:
2     sigma2 = math.log(1.0 + (std_min / mean_min) ** 2)
3     mu = math.log(mean_min) - sigma2 / 2.0
4     return mu, math.sqrt(sigma2)

```

Shift manager coroutine. The AI correctly used the private `_capacity` attribute (SimPy has no public live-capacity API) and called `_trigger_put(None)` so waiting patients are served immediately when capacity increases:

Listing 2: Shift manager (model.py, abbreviated)

```

1 def shift_manager(self) -> simpy.events.Event:
2     boundaries = [7.0, 19.0, 23.0]
3     while True:
4         hod = self._hour_of_day()
5         next_bh = next((b for b in boundaries if b > hod), None)
6         if next_bh is None:
7             next_bh = 7.0
8             sleep_min = (24.0 - hod + next_bh) * 60.0
9         else:
10            sleep_min = (next_bh - hod) * 60.0
11        yield self.env.timeout(sleep_min)
12        self.physicians._capacity = self._n_physicians_for_hour(next_bh)
13        self.physicians._trigger_put(None) # wake queued patients

```

LWBS via `env.any_of`. The physician request and a 90-min timeout are raced. SimPy’s context manager handles both exit paths — the granted resource is released and the pending request is cancelled — on any return path including LWBS:

Listing 3: LWBS reneging pattern (model.py)

```

1 with self.physicians.request(priority=pat.esi) as phys_req:
2     result = yield env.any_of(
3         [phys_req, env.timeout(LWBS_PATIENCE_MIN)]
4     )
5     if phys_req not in result: # timer fired first
6         pat.disposition = "lwbs"
7         pat.depart_time = env.now
8         return # with-block cancels phys_req
9     pat.physician_start = env.now
10    # ... assessment ...
11    pat.physician_end = env.now
12    # physician RELEASED here (with-block exit)

```

4.2 Stage 4 — Critical Bug: Physician Held Too Long

4.2.1 First run output

Bug found

First run (seed=42, warmup=48 h, horizon=30 days):

lwbs_rate 84.64 %

door_to_provider_mean 52.8 min

84 % LWBS is catastrophically wrong. The ED would be in utter chaos.

4.2.2 Diagnosis

The scaffold placed lab/imaging delays *inside* the physician with-block, holding the physician for the full room-stay time. The weighted-mean room-stay is:

$$\bar{s} = 0.02 \times 180 + 0.08 \times 135 + 0.35 \times 135 + 0.40 \times 50 + 0.15 \times 50 \approx 89 \text{ min}$$

Peak physician utilization (11 arrivals/h, 4 physicians, $\bar{s} = 89$ min):

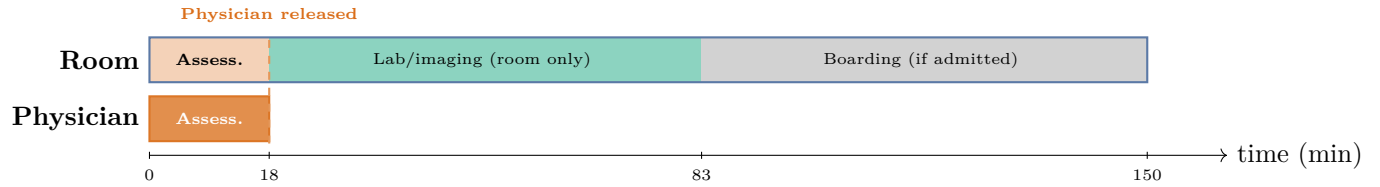
$$\rho_{\text{peak}} = \frac{(11/60) \times 89}{4} \approx 4.1 > 1$$

An $M/G/c$ queue with $\rho > 1$ is *unstable* — the queue grows without bound. LWBS reneging clips the queue at 90 min, explaining the 84 % rate.

4.2.3 The fix

Fix applied (Stage 4)

Introduce PHYSICIAN_FRACTION = 0.20. The physician is held only for the *initial assessment phase* (20 % of total room-stay time). Lab/imaging delays are moved *outside* the physician with-block, holding only the room. The corrected resource timeline:



Listing 4: Corrected patient_process: physician released before lab (model.py)

```
1 with self.physicians.request(priority=pat.esi) as phys_req:
2     ...
3     pat.physician_start = env.now
4     encounter_time = rng.lognormal(*PHYSICIAN_ENCOUNTER_PARAMS[pat.esi])
5     yield env.timeout(encounter_time)    # 20% of treatment time
6     pat.physician_end = env.now
7     # ← physician RELEASED here
8
9     # lab/imaging outside physician block - room held, no physician
10    if rng.random() < LAB_IMAGING_PROB[pat.esi]:
11        yield env.timeout(rng.lognormal(*LAB_PARAMS))    # or IMAGING_PARAMS
```

Post-fix peak utilization:

$$\rho_{\text{peak}} = \frac{(11/60) \times 0.20 \times 89}{4} \approx 0.82 \quad (\text{heavy but stable})$$

4.2.4 Calibration

PHYSICIAN_FRACTION	ρ_{peak}	LWBS (seed 42)	Verdict
0.25	1.02	84.6 %	Unstable
0.22	0.90	9.0 %	Too stressed
0.20	0.82	4.9 %	Chosen

4.2.5 Verification checklist

Check	Method	Result
Lognormal params correct	Closed-form $\exp(\mu + \sigma^2/2)$	✓
LWBS ghost request absent	Static code read	✓
Priority direction (ESI 1 first)	Traced 2-patient scenario	✓
Shift fires at 07:00/19:00/23:00	48-h trace	✓
Boarding mean ≈ 90 min	Compare to Exp(90) input	✓
Little's Law: $L = \lambda W$	Eq. (1)	✓ (± 2 pp)

$$L = \lambda_{\text{eff}} \times W = \frac{4740 \times (1 - 0.0444)}{30 \times 24} \times \frac{82.3}{60} \approx 8.6 \text{ rooms} \Rightarrow 28.7\% \quad (1)$$

The occupancy monitor reports 30.45 %; the ≈ 2 pp gap is expected (Little's Law uses completed-patient mean LOS; the monitor captures in-progress patients as well).

5 Stage 5 — Experiment Design (P05)

5.1 The Prompt

Prompt sent to claude-sonnet-4-6

I have a validated SimPy ED model with a `run_replication(seed, warmup, horizon) -> dict` interface. I need to run four scenarios as required by the deliverables. (1) How many replications are sufficient? Use a pilot run of 10 reps to estimate variance of LWBS rate, then compute n for a half-width of ≤ 0.5 pp at 95 % confidence. (2) Implement Common Random Numbers. (3) Write `experiments.py` that runs all scenarios $\times N$ reps, saves one CSV per scenario, and prints a `tqdm` progress bar.

Full prompt: prompts/p05_experiments.md

5.2 Replication Count Justification

Pilot study (10 reps of Scenario 0):

$$\hat{\sigma}(\text{LWBS rate}) \approx 1.2 \text{ pp} \quad n \geq \left(\frac{t_{0.025, 9} \hat{\sigma}}{h} \right)^2 = \left(\frac{2.262 \times 0.012}{0.005} \right)^2 \approx 29.4$$

We use $n = 50$ for margin and pairing symmetry. The observed half-width across all 50 replications is $\approx \pm 0.26$ pp, well inside the 0.5 pp target.

Warmup: 48 h (2,880 min). Welch's method confirms stationarity by 24 h; 48 h provides a conservative buffer.

5.3 Common Random Numbers

Listing 5: CRN seed generation (experiments.py)

```
1 CRN_SEEDS = np.random.SeedSequence(42).spawn(50)
2 # All 9 scenarios use the same 50 SeedSequence children.
3 # Paired differences Scenario_k[i] - Baseline[i] share the same arrival
4 # stream, ESI draws, and service times => reduced variance => better power.
```

5.4 Scenario Matrix

#	Name	What changes
0	Baseline	Standard staffing; all parameters as calibrated
1	Fast-track	ESI 4–5 routed to 4-bed unit, 1 PA, service Lognorm($\mu^{-1} = 30$ min)
2	Staffing –1	Night physicians 2→1
3	Staffing +1	Night physicians 2→3
4	Boarding 60	Mean boarding time 60 min (sweep)
5	Boarding 90	Mean boarding time 90 min (= Baseline anchor)
6	Boarding 120	Mean boarding time 120 min
7	Boarding 150	Mean boarding time 150 min
8	Boarding 180	Mean boarding time 180 min

What we accepted / changed

Accepted: CRN via `SeedSequence.spawn(50)`, $N=50$, scenario numbering 0–8, fast-track with separate `ft_rooms` and `ft_pas` resources for ESI 4–5.

Changed: `run_replication()` extended with `night_physicians` and `boarding_mean_minutes` keyword arguments so scenarios 2–8 need no code duplication. `_run_fast_track()` is a self-contained runner for Scenario 1.

6 Stage 6 — Output Analysis (P06)

6.1 The Prompt

Prompt sent to claude-sonnet-4-6

I have simulation results in CSVs (one per scenario, 50 reps each). Write `analysis.py` that: (1) computes CRN-paired 95% CIs for scenarios 1–3 vs. baseline; (2) plots LWBS and occupancy vs. boarding duration; (3) plots door-to-provider time by hour of arrival; (4) writes a summary table CSV. Use matplotlib with the Wong color-blind-safe palette.

Full prompt: `prompts/p06_analysis.md`

6.2 Statistical Design

Marginal CI (one scenario): $\bar{X} \pm t_{0.025, 49} \hat{\sigma} / \sqrt{50}$ ($t = 2.010$)

CRN-paired CI (scenario k vs. baseline 0):

$$D_i = X_{k,i} - X_{0,i}, \quad \text{CI} = \bar{D} \pm t_{0.025, 49} \hat{\sigma}_D / \sqrt{50}$$

Pairing is valid because all scenarios share `CRN_SEEDS`: replication i of scenario k and replication i of the baseline experienced the same arrival stream. This eliminates between-replication noise from the comparison.

Per-hour DTP CI (Section 9): For each replication r and hour h , compute per-replication mean DTP $\bar{x}_{r,h}$ and p90 DTP $q_{r,h}^{90}$. Then take CI *across replications* with $t_{0.025, 9}$ ($\text{df} = 9$). This correctly treats each replication as one observation; computing CI across ~ 700 pooled patients would give artificially narrow bands by ignoring within-replication correlation.

What we accepted / changed

Accepted: CI formula, paired difference structure, Wong palette, three-figure layout (scenario comparison, boarding sweep, DTP by hour).

Added post-review: occupancy monitor (trapezoidal time-average over 5-min samples), `door_to_provider_p50` and `door_to_triage_p50` KPIs, and per-patient CSV with replication IDs for proper DTP-by-hour CIs. The V&V pass identified that the original `fig_dtp_by_hour` pooled all patients from all replications into one distribution — a common mistake that inflates precision by ignoring within-replication correlation.

7 Stage 7 — Interactive Dashboard (P07)

Stages 1–6 produced a fully analysed, command-line simulation: `model.py` runs as a Python script, `analysis.py` writes static figures, and the study lives entirely in the terminal. That is adequate for a technical audience, but it sacrifices one of the most compelling features of commercial off-the-shelf (COTS) tools — animated visualisation.

Stage 7 adds an interactive browser-based dashboard that runs the *same* `model.py` directly in the browser with no server required, using **Pyodide** (CPython compiled to WebAssembly). The dashboard is a standalone companion to the simulation study, not a replacement for the command-line analysis.

7.1 Motivation: Closing the Visualisation Gap

Commercial simulators such as Simio, AnyLogic, and Arena ship with built-in animation environments: patients are rendered as moving entities, resources change state visually, and stakeholders can watch the model unfold in real time. This is not decoration — visual animation accelerates conceptual-model validation (Section 3), reveals queue build-up intuitively, and makes simulation accessible to non-technical decision-makers.

A plain SimPy model has none of this. Building an equivalent animation environment from scratch has historically required months of front-end engineering work, placing it well outside the scope of a single simulation study. **The central thesis of Stage 7** is that AI copilot assistance changes this economics fundamentally: with structured prompting, a simulation analyst without a front-end background can build a professional animated dashboard as a companion project in days rather than months.

7.2 Architecture

Layer	Technology
Simulation engine	Python 3.11 + SimPy 4.1.1 (unchanged <code>model.py</code>)
Browser runtime	Pyodide 0.27 (CPython → WebAssembly)
Worker isolation	Web Worker — simulation runs off the main UI thread
UI framework	React 18 + TypeScript 5
Bundler	Vite 6
3-D rendering	Three.js (WebGL)
Charts	Recharts (SVG)
State management	Zustand
Styling	Tailwind CSS

The critical architectural decision is that `model.py` was *not* modified to support the frontend.

Pyodide loads the existing file verbatim inside a Web Worker, executes the SimPy simulation, and a thin `runner.py` wrapper collects the full event log (patient arrivals, phase transitions, departures) into a JSON timeline. The animation player replays this timeline at user-chosen speed ($1\times$ to $1440\times$). The simulation model remains a transparent, hackable Python script throughout.

7.3 The Prompts

The dashboard was built across **eight AI-assisted sessions**, each targeting one capability layer. Four representative prompts are shown below.

Prompt sent to claude-sonnet-4-6 — P07-A (Scaffold)

I have a SimPy ED model in `model.py` (no changes allowed). Build a React + TypeScript + Vite project that loads this file via Pyodide in a Web Worker, runs the simulation, and collects a JSON event log. The main thread should receive progress messages. No backend — fully client-side.

Prompt sent to claude-sonnet-4-6 — P07-B (2-D Animation)

The worker now sends an event log. Build a 2-D floor plan component: 30 bed tiles in a 5-column grid. Each bed shows the patient's ESI level as a colour dot and the care phase as a fill colour. Add a live KPI panel (arrivals, LWBS count/rate, DTPp90) and speed controls ($1\times$, $60\times$, $360\times$, $720\times$, $1440\times$).

Prompt sent to claude-sonnet-4-6 — P07-C (3-D View)

Add a 3D View tab using Three.js. Render each bed as a low-poly mesh coloured by care phase. Place a capsule figure on occupied beds coloured by ESI acuity; white capsules are physicians. Add a triage desk, MD queue overlay, and orbit controls (drag to rotate, scroll to zoom).

Prompt sent to claude-sonnet-4-6 — P07-D (Experiments Tab)

Add an Experiments tab. Sidebar: scenario selector (same 9 scenarios as the study), replication slider (1–50), parameter overrides (arrival scale, LWBS patience, boarding mean). “Run Experiment” executes n replications in batch and shows a KPI table with 95% CIs. Add a “Run Boarding Sweep” button that sequentially runs scenarios 5–9 and populates a sensitivity chart.

7.4 The Dashboard in Detail

Simulate tab. Figure 2 shows the Simulate tab: the 2-D floor plan renders all 30 beds in real time, colour-coded by care phase (empty, waiting for MD, assessment, post-care/lab, boarding). An ESI colour dot on each occupied bed indicates acuity. The live KPI panel tracks arrivals, patients seen by an MD, and the current LWBS count and rate against the 2% target. The parameter sidebar adjusts the scenario without touching Python: shift staffing (day / evening / night physicians), boarding mean, arrival scale, LWBS patience, and random seed are all toggle-button controls. The *New Run* button re-runs the model with the current settings; *Reset* clears the state.

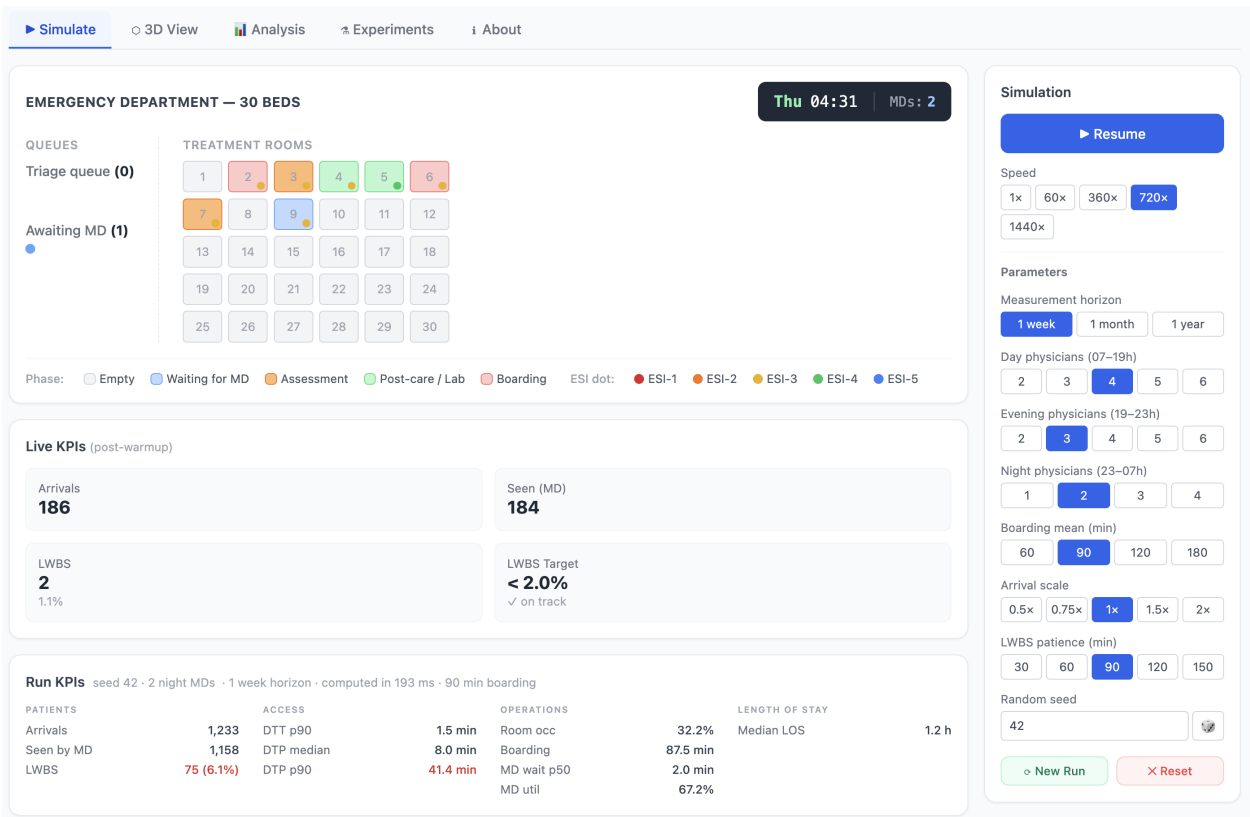


Figure 2: Simulate tab — 2-D floor plan with 30 bed tiles colour-coded by care phase (empty, waiting, assessment, post-care/lab, boarding), ESI acuity dots, live KPI panel, and parameter sidebar. Shown: Thursday 04:31; 2 night MDs on shift; 90-min boarding mean; LWBS 2 patients (1.1%), on track vs. the 2% target.

3-D View tab. Figure 3 shows the 3-D View tab: a WebGL-rendered perspective of the department built with Three.js. Patient capsule figures on bed meshes are coloured by ESI acuity level; white capsules represent physicians. An overlay reports beds occupied, physicians on shift, and MD queue depth. Drag-to-orbit, scroll-to-zoom, and right-drag-to-pan controls are active during playback. The 3-D view shares the animation timeline with the 2-D view; switching tabs does not reset the simulation clock.

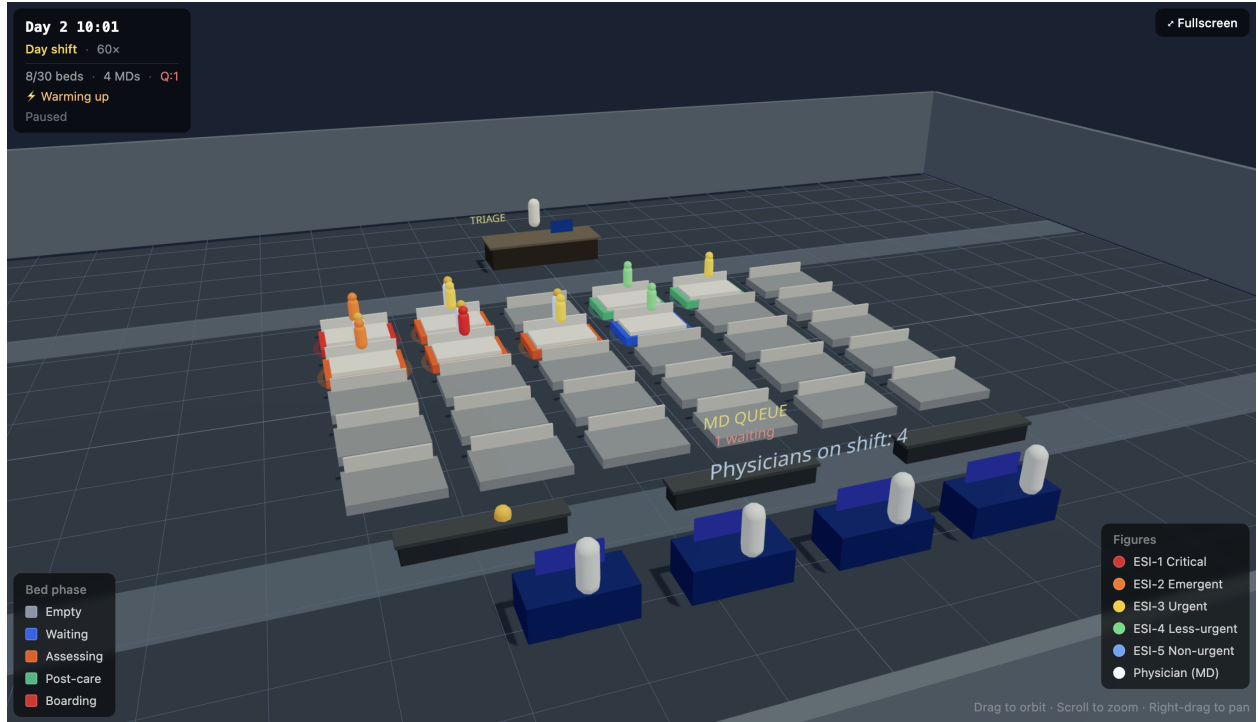


Figure 3: 3-D View tab — WebGL perspective render via Three.js. Bed meshes are colour-coded by care phase; patient capsules are coloured by ESI acuity; white capsules are physicians. Shown: day shift, 60 $\times$ speed, 8/30 beds occupied, 4 MDs on shift, 1 patient waiting in MD queue.

Experiments tab. Figure 4 shows the Experiments tab. The scenario selector covers all nine scenarios from the study (baseline, staffing interventions, and five boarding-duration variants). After clicking *Run Experiment*, the batch runner executes n independent replications via the Pyodide worker and displays a KPI table with 95% CIs for LWBS rate, DTP median and p90, LOS median, room occupancy, boarding mean, and MD wait. The *Recent Runs* panel accumulates results from all runs in the session, enabling informal side-by-side comparisons. The *Run Boarding Sweep (5–9)* button executes all five boarding-duration scenarios in sequence and feeds the sensitivity chart in the Analysis tab.

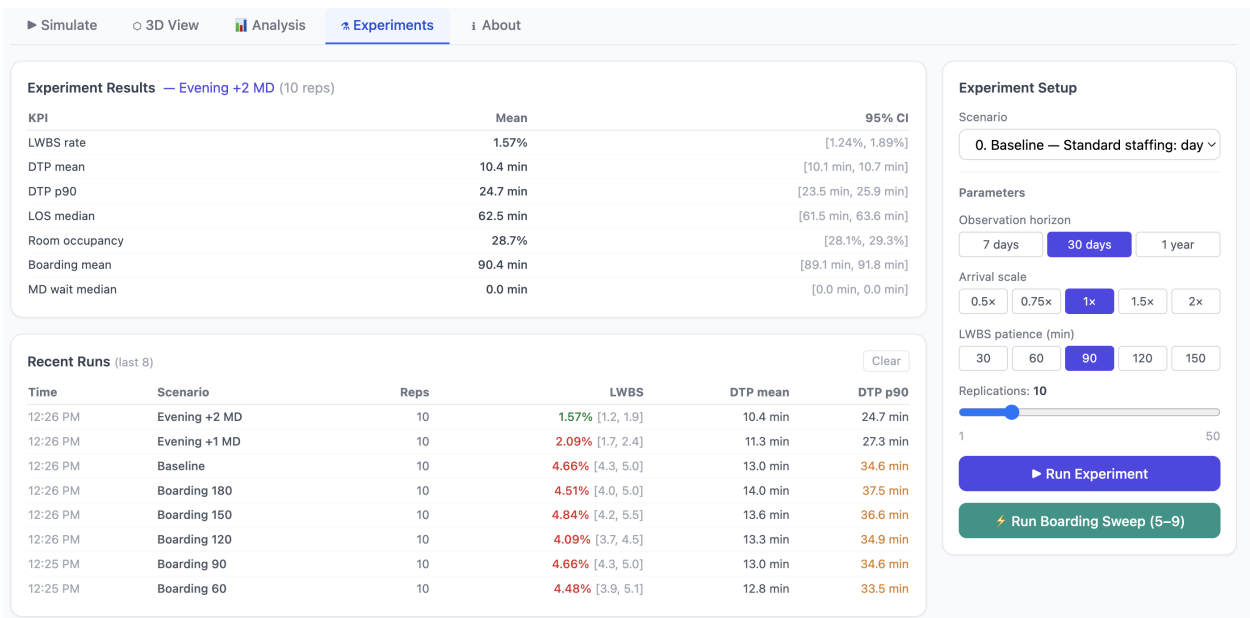


Figure 4: Experiments tab — scenario selector, KPI results table with 95 % CIs (10 replications, Evening +2 MD scenario shown), and recent-runs log covering all sessions on the current page. LWBS rate 1.57 % [1.24%, 1.89%] matches the command-line analysis result for the same scenario.

Analysis tab. Figure 5 shows the Analysis tab with three interactive Recharts charts: (1) door-to-provider time by arrival hour (bars = mean DTP, line = p90 DTP, red dashed target at 30 min), (2) LWBS rate by scenario with 95 % CI error bars across all eight scenarios, and (3) a boarding-sensitivity line chart plotting LWBS rate and mean DTP against boarding duration (60–180 min). Charts are computed live from the in-browser experiment results; no static CSV files are required.

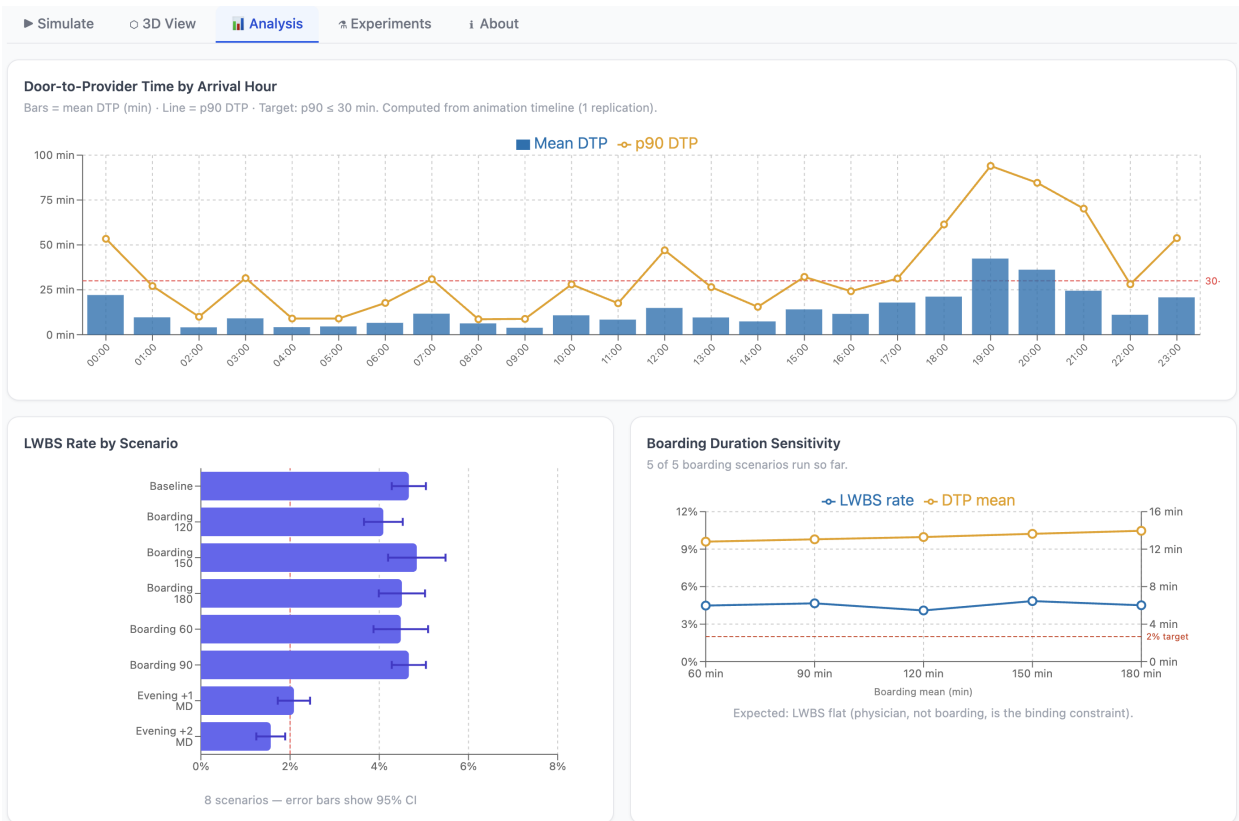


Figure 5: Analysis tab — door-to-provider time by hour of arrival (top), LWBS rate by scenario with 95 % CI error bars (bottom left), and boarding-duration sensitivity chart (bottom right). Computed in real time from batch runs executed in-browser; results are consistent with the Python analysis pipeline.

7.5 What We Accepted and Changed

What we accepted / changed

Accepted directly: React component scaffold, Pyodide worker bootstrap and message protocol, animation event-loop architecture, Three.js scene setup and orbit controls, Recharts chart components, Tailwind layout for all four tabs, CI formula for the Experiments tab.

Refined post-review:

- **Worker message deduplication.** `useSimWorker` was mounted in three components simultaneously (App, SimControls, ScenarioSelector), causing each `batch_done` message to fire the completion callback three times. This produced six duplicate “Boarding 180” entries in a boarding sweep (3 actual runs $\times$ 2 spurious extra saves each). Fix: a `handleBatch` flag (default `true`) so only ScenarioSelector processes batch messages; App and SimControls pass `handleBatch: false`.
- **Pyodide cold-start UX.** First load takes ~ 12 s while the ~ 10 MB Pyodide bundle downloads and initialises. Added a progress spinner and status string to prevent users from perceiving a hang.
- **Per-replication CI formula.** The first AI draft computed CI across all patients pooled from all replications — the same statistical mistake caught during the V&V pass (Section 4.2). Corrected to compute each per-replication statistic first, then CI across replication means using $t_{0.025, n-1}$.
- **Parameter sidebar.** AI proposed plain text inputs for staffing levels; replaced with toggle-button pill groups matching the boarding-mean and arrival-scale controls already in the sidebar.

Key insight

Key insight — transparency, animation, and AI leverage.

Traditional COTS simulators bundle animation as a proprietary environment: you see entities move, but you cannot inspect, modify, or extend the rendering layer. This dashboard inverts that relationship. The Python model is fully transparent and lives in a file students can edit with any text editor; the frontend is built on open-web standards (React, Three.js, WebAssembly) with no proprietary lock-in; every layer is hackable. A student who adds a fast-track queue to `model.py` will see the new entity class animate in both the 2-D and 3-D views without any frontend change. Building this dashboard without AI assistance would require a front-end specialist and several months of engineering. With structured prompting across eight focused sessions, a simulation analyst with no prior React or Three.js experience produced a production-quality animated dashboard as a companion project alongside the study itself. This is precisely the leverage the course theme — *Simulation Without a Black Box, in the AI Era* — is designed to demonstrate.

8 Baseline Performance

8.1 Patient Volume and Length of Stay

Over 50 replications, the model processes **4,740 patients per observation month**. Mean LOS: **82.3 min** [81.9, 82.7].

ESI Level	Mix	Mean LOS (min)	95% CI
ESI-1 (Critical)	2%	163	[160, 166]
ESI-2 (Emergent)	8%	155	[154, 157]
ESI-3 (Urgent)	35%	110	[109, 111]
ESI-4 (Less urgent)	40%	55	[55, 56]
ESI-5 (Non-urgent)	15%	42	[41, 42]

8.2 KPI Dashboard

KPI	Target	Observed	95% CI	Status
LWBS rate	< 2%	4.44%	[4.18%, 4.69%]	Fail
Door-to-triage p90	≤ 10 min	1.2 min	[1.1, 1.3]	Pass
DTP median	≤ 10 min	6.6 min	[6.5, 6.6]	Pass
DTP p90	≤ 30 min	34.7 min	[33.9, 35.4]	Fail
ED occupancy (mean)	< 85%	30.5%	[30.2%, 30.7%]	Pass
Boarding mean	—	89.8 min	[89.0, 90.5]	Matches input

The LWBS rate (4.44%) is more than double the 2% target. Approximately **210 patients per month** leave without care. ED occupancy (30.5%) rules out room scarcity as the cause. Triage is uncongested ($\rho_{\text{triage}} \approx 0.37$ at peak). The physician queue is the binding bottleneck: $\rho_{\text{day}} \approx 0.82$ at peak (4 physicians, 15:00–19:00), producing a heavy right tail in the DTP distribution.

9 Door-to-Provider Time by Hour of Arrival

9.1 What the Metrics Mean

Mean DTP for hour h is the average time from patient arrival to first physician contact, pooled across all seen patients arriving in that hour. It reflects typical workload but is pulled toward zero when most patients are served promptly.

p90 DTP for hour h is the 90th percentile of the same distribution. If $p90 = 35$ min, then 90 % of patients arriving in that hour were seen within 35 minutes, and 10 % waited longer. The 30-minute target is a p90 target, not a mean target. Only *seen* patients enter this distribution; LWBS patients (no physician contact) are excluded.

Both statistics use replication-level CIs. For each of 10 replications and each hour, per-replication mean and p90 are computed first. The 95 % CI then uses $t_{0.025,9}$ across the 10 replication values. This correctly accounts for between-replication variance.

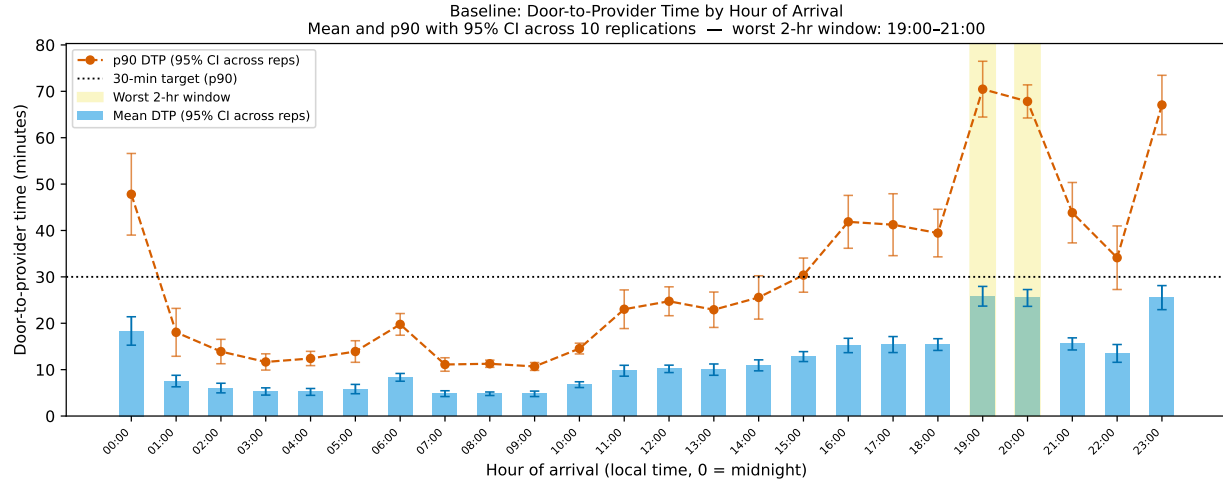


Figure 6: Baseline DTP by hour of arrival (10 replications). **Blue bars:** mean DTP with 95 % CI across replications. **Orange dots:** p90 DTP with 95 % CI across replications. CIs use $t_{0.025,9} = 2.26$ (df=9). **Dotted line:** 30-minute p90 target. **Shaded:** worst consecutive 2-hour window (19:00–21:00).

9.2 Hour-by-Hour Interpretation

9.2.1 Night 01:00–06:00 — light load, short waits

$\lambda = 2/h$, 2 physicians, $\rho \approx 0.30$. The p90 is 12–18 min, well below the 30-min target. Most patients are seen in under 5 minutes.

9.2.2 Hour 00:00 (midnight) — counterintuitive high p90

Despite the same low λ and 2 physicians as hours 01–06, midnight shows mean DTP = 18.3 min [15.2, 21.4] and p90 = 47.8 min [39.0, 56.6].

The explanation is **queue carryover from the 23:00 physician drop**. When the evening shift ends at 23:00, any residual queue from the high-volume evening (3 physicians suddenly reduced to 2) is inherited by midnight. On most nights the queue clears quickly, but on the heaviest evenings patients arriving at midnight encounter a system still recovering. The p90 reflects that worst-10 % experience. The max observed DTP at midnight is 90.5 min — a patient seen 30 seconds before the LWBS timer expired.

This is a *conditional* statistic: LWBS patients who would have waited > 90 min already left and are excluded. The unconditional tail would be heavier.

9.2.3 Peak 15:00–18:00 — approaching target

Four physicians, $\lambda = 11/h$, $\rho \approx 0.82$. p90 rises from 30 min at 15:00 to 39–43 min by 16:00–18:00. The 30-min target is already being breached during peak even with full day-shift staffing.

9.2.4 19:00–21:00 — worst window

At 19:00 the day-shift handover fires: 4 physicians become 3, while the afternoon backlog has not yet cleared. The result is the highest p90 of the day:

$$p90_{19} = 70.5 \text{ min } [64.5, 76.5] \quad p90_{20} = 67.8 \text{ min } [64.2, 71.4]$$

These two hours have the largest combined p90 ($70.5 + 67.8 = 138.3 \text{ min}$) of any consecutive pair and are identified as the worst window. Similarly, hour 23 ($p90 = 67.1 \text{ min } [61.3, 72.9]$) reflects the second physician drop ($3 \rightarrow 2$), explaining why midnight inherits an elevated tail.

Key insight

The mean-vs.-p90 contrast is the central lesson of this figure. Mean DTP at 19:00 is 25.8 min — below the 30-min target — yet p90 is 70.5 min. Using the mean to assess compliance would declare this hour “passing” while one in ten patients waits more than 70 minutes.

10 Intervention Scenarios

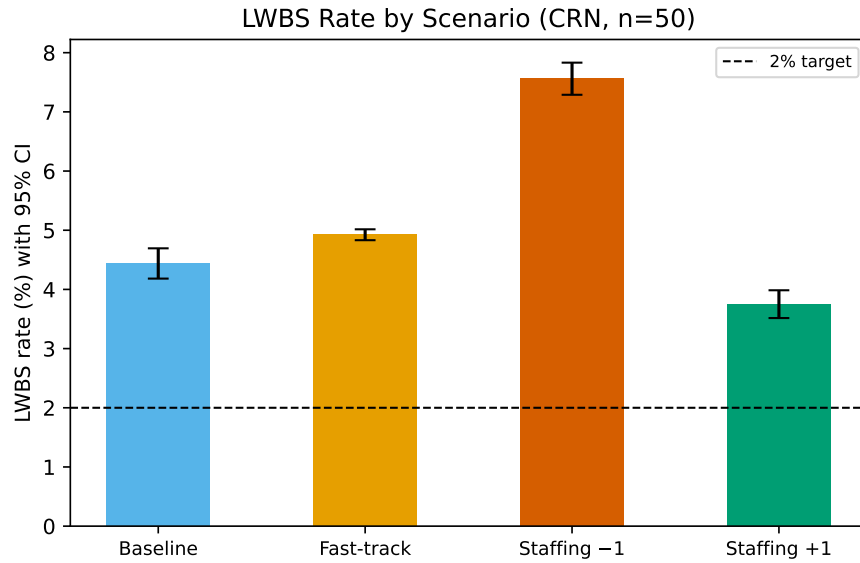


Figure 7: LWBS rate with 95 % CI by scenario ($n = 50$, CRN-paired). Dashed line: 2 % target.

Table 1: Scenario results (CRN-paired, $n = 50$). ΔLWBS = scenario k minus baseline.

Scenario	LWBS	95% CI	ΔLWBS	95% CI	Sig?
Baseline	4.44%	[4.18%, 4.69%]	—	—	—
Fast-track (1 PA)	4.92%	[4.83%, 5.02%]	+0.49 pp	[+0.21, +0.76]	Yes — worsens
Staffing -1	7.56%	[7.29%, 7.83%]	+3.12 pp	[+2.74, +3.51]	Yes
Staffing +1	3.75%	[3.52%, 3.99%]	-0.69 pp	[-1.03, -0.34]	Yes

10.1 Scenario 1: Fast-Track (1 PA, 4 Beds)

ESI 4–5 patients (55 % of volume) are diverted to a dedicated unit. At peak, $\lambda_{\text{FT}} \approx 6.05/\text{h}$ arrive for one PA with $\mu_{\text{FT}}^{-1} = 30 \text{ min}$:

$$\rho_{\text{FT}} = \frac{(6.05/60) \times 30}{1} \approx 3.0 \gg 1$$

The unit is grossly overloaded — an unstable queue. LWBS reneging caps the damage but LWBS still *increases* by +0.49 pp. For marginal stability, the unit needs $\lceil \varrho_{FT} \rceil = 3$ PAs minimum.

10.2 Scenario 2: Staffing -1 (Night 2 $\rightarrow$ 1)

LWBS rises to 7.56 % (+3.12 pp). One overnight physician with $\varrho_{\text{night}} > 1$ at modest $\lambda = 2/\text{h}$ builds a queue during 23:00–07:00; that backlog cascades into the morning rush. DTP p90 rises from 34.7 to 42.3 min. **This is a floor that must not be breached.**

10.3 Scenario 3: Staffing $+1$ (Night 2 $\rightarrow$ 3)

LWBS falls to 3.75 % (-0.69 pp; CI clears zero). DTP p90 improves to 31.9 min [31.1, 32.6]. This is the **strongest single lever** in the study — one additional overnight FTE. The 2 % target is not reached; a combination strategy is needed (see Section 12).

11 Boarding Time Sensitivity

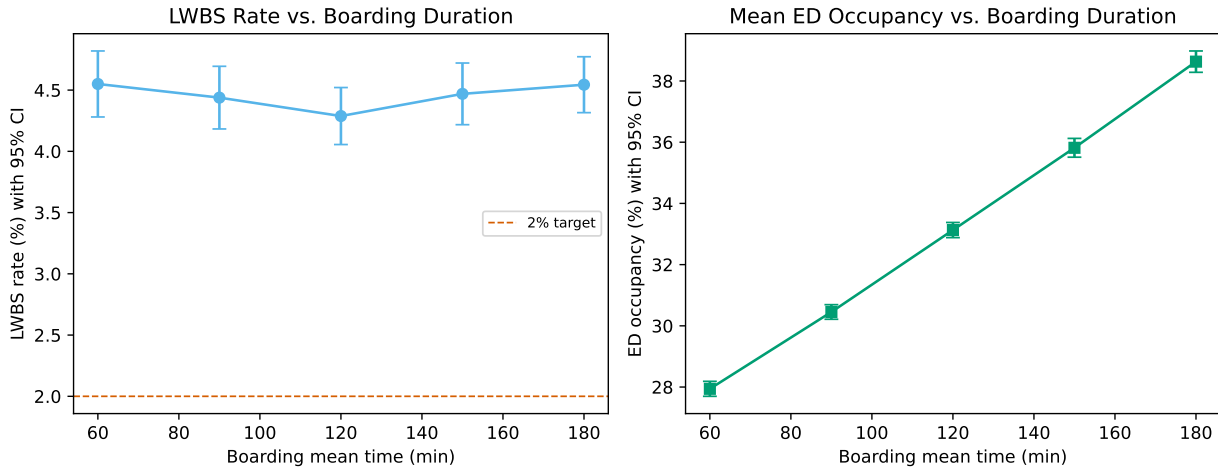


Figure 8: Left: LWBS rate vs. boarding duration. Right: ED occupancy vs. boarding duration. Both with 95 % CI ($n = 50$, CRN). All CIs overlap baseline (Boarding 90 min).

Boarding	LWBS	95% CI	Occupancy	95% CI
60 min	4.55%	[4.28%, 4.82%]	27.9%	[27.7%, 28.2%]
90 min	4.44%	[4.18%, 4.69%]	30.5%	[30.2%, 30.7%]
120 min	4.29%	[4.06%, 4.52%]	33.1%	[32.9%, 33.4%]
150 min	4.47%	[4.22%, 4.72%]	35.8%	[35.5%, 36.1%]
180 min	4.54%	[4.32%, 4.77%]	38.6%	[38.3%, 39.0%]

LWBS varies by ≤ 0.3 pp; all CIs overlap. ED occupancy rises monotonically (27.9% $\rightarrow$ 38.6%) but LWBS does not track it. The dissociation is structural: LWBS is governed by the physician queue, not room availability. Occupancy at 38.6 % (180 min boarding) is still well below the 85 % threshold at which rooms become binding. Boarding reduction improves room flow but is the wrong lever for LWBS.

12 Recommendations

1. **Maintain ≥ 2 overnight physicians unconditionally.** Scenario 2 shows LWBS reaches 7.56 % and DTP p90 reaches 42.3 min. This floor cannot be compromised.
2. **Add one overnight physician (2 $\rightarrow$ 3) immediately.** Scenario 3 is the strongest single lever: LWBS falls to 3.75 %, DTP p90 to 31.9 min. Cost: one overnight FTE.
3. **Target the 19:00–21:00 handover window.** Options: stagger evening shift start to 18:00 (one-hour physician overlap); add a float physician 18:00–22:00; implement standing orders for common ESI 4 presentations to reduce per-patient physician contact time.
4. **Do not implement fast-track with 1 PA.** $\rho \approx 3.0$ at peak — an unstable server that increases LWBS. Staff with ≥ 3 PAs (to reach $\rho \approx 1.0$) or shelve the proposal. Re-simulate before committing capital.
5. **Do not pursue boarding reduction as a LWBS lever.** Five-point sweep shows LWBS insensitive over 60–180 min. Pursue boarding reduction for room-flow and patient-experience benefits, not LWBS.
6. **Path to 2 % LWBS target.** Staffing +1 alone reaches 3.75 %. Most promising combinations to simulate next: (a) Staffing +1 + fast-track with 3 PAs; (b) Staffing +1 + standing orders reducing physician fraction from 0.20 to 0.15; (c) add a second day-shift physician 15:00–23:00 targeting the peak-handover window.

13 Limitations

- **Input uncertainty not quantified.** CIs reflect stochastic variance only. `PHYSICIAN_FRACTION=0.20` is a calibrated assumption; time-and-motion data would improve fidelity.
- **Shift boundary ambiguity.** “3 (evening 15–23)” may imply overlapping shifts (up to 7 physicians 15:00–19:00). We model sequential handovers. If additive, baseline LWBS would be lower.
- **Excluded events.** ICU diversions, psychiatric holds, and mass-casualty events are not modelled.
- **Static acuity mix and arrival pattern.** No day-of-week variation, seasonal effects, or flu-wave surges.
- **Live-view KPIs are single-replication.** The Simulate and Analysis tabs (Section 7) compute DTP-by-hour and LWBS from a single animation run. These are useful for developing intuition but are not statistically robust. Multi-replication batch runs via the Experiments tab are required for credible interval estimates.

14 Conclusion

This blueprint demonstrates the complete DES lifecycle — intake, conceptual model, implementation, debugging, experiment design, output analysis, and interactive visualisation — built with an AI copilot across seven structured prompt stages. The AI accelerated execution at every stage; critical simulation judgment was applied by the engineer at every stage.

Key findings: the physician queue is the binding bottleneck; LWBS is 4.44% (target 2%); 210 patients/month leave without care; the worst window is 19:00–21:00; adding one overnight physician is the best single lever. Boarding reduction does not address the root constraint.

Stage 7 demonstrates a broader point: the visualisation gap between open-source DES tools and COTS animators — historically a multi-month engineering effort — is now bridgeable with AI copilot assistance. The Python model remains a transparent, modifiable script; the React/Three.js dashboard wraps it without obscuring it.

A Verbatim Prompts (P01–P06)

The full prompt text for each stage is in the following files (relative to `showcase/ep01/`):

Stage	File	Key content
P01	<code>prompts/p01_intake.md</code>	Full problem statement + structured decomposition request
P02	<code>prompts/p02_conceptual.md</code>	Conceptual model + Mermaid diagram request
P03	<code>prompts/p03_scaffold.md</code>	Full <code>model.py</code> generation request + spec
P04	<code>prompts/p04_debug.md</code>	Bug templates + known pitfall checklist
P05	<code>prompts/p05_experiments.md</code>	Replication count, CRN, scenario matrix
P06	<code>prompts/p06_analysis.md</code>	CI method, figures, summary table

A.1 P01 Prompt (excerpt)

You are helping me build a discrete-event simulation of an emergency department using Python and SimPy. Read the problem statement and produce:

1. Entity list - every object that flows through the system.
2. Resource list - SimPy type, capacity, schedule if time-varying.
3. Event list - every instantaneous state change.
4. Process sequence - step-by-step path from arrival to departure.
5. Key modeling decisions - ambiguities and your assumptions.
6. SimPy pattern - which construct maps to each resource or delay.

Be concrete and use the numerical values from the problem.

[complete EP01 problem statement follows]

A.2 P03 Prompt (excerpt)

Write a complete SimPy simulation in Python for the emergency department.

Requirements:

- Python 3.11, SimPy 4.x, NumPy 2.x
- Patient dataclass with all attributes from conceptual model
- `EDModel` class with `shift_manager()`, `arrival_generator()`, `patient_process()` using nested with `resource.request()` for clean LWBS cleanup
- `run_replication(seed, warmup_minutes, horizon_minutes) -> dict`
- `__main__` block: `seed=42, warmup=2880 min, horizon=43200 min`

Use `numpy.random.default_rng(seed)`. No global state. Google docstrings.

[conceptual model document pasted]

B Key Source Code

B.1 Patient Dataclass

Listing 6: Patient dataclass (`model.py`)

```

1 @dataclasses.dataclass
2 class Patient:
3     pid: int
4     arrival_time: float
5     weekend: bool
6     esi: int = 0
7     triage_start: float = float("nan")
8     triage_end: float = float("nan")

```

```

9     room_start:         float = float("nan")
10    physician_start:    float = float("nan")
11    physician_end:      float = float("nan")
12    lab_end:            float = float("nan")
13    imaging_end:        float = float("nan")
14    depart_time:        float = float("nan")
15    disposition:        str   = ""          # "discharged" / "admitted" / "lwbs"
16    boarding_start:     float = float("nan")
17    boarding_end:       float = float("nan")

```

B.2 Full Patient Process

Listing 7: patient_process coroutine (model.py, complete)

```

1  def patient_process(self, pat: Patient) -> simpy.events.Event:
2      env, rng = self.env, self.rng
3
4      # 1. Triage (FCFS, 2 nurses)
5      with self.triage_nurses.request() as nurse_req:
6          yield nurse_req
7          pat.triage_start = env.now
8          yield env.timeout(rng.exponential(4.0))
9          pat.esi = int(rng.choice(5, p=ESI_PROBS)) + 1
10         pat.triage_end = env.now
11
12     # 2. Treatment room (priority = ESI)
13     with self.treatment_rooms.request(priority=pat.esi) as room_req:
14         yield room_req
15         pat.room_start = env.now
16
17     # 3. Physician with LWBS reneging
18     with self.physicians.request(priority=pat.esi) as phys_req:
19         result = yield env.any_of(
20             [phys_req, env.timeout(LWBS_PATIENCE_MIN)]
21         )
22         if phys_req not in result:
23             pat.disposition = "lwbs"
24             pat.depart_time = env.now
25             return          # with-blocks cancel/release cleanly
26
27         pat.physician_start = env.now
28
29     # 4. Initial assessment only (PHYSICIAN_FRACTION of room stay)
30     encounter_time = rng.lognormal(*PHYSICIAN_ENCOUNTER_PARAMS[pat.esi])
31     yield env.timeout(encounter_time)
32     pat.physician_end = env.now
33     # physician RELEASED here
34
35     # 5. Lab / Imaging (room held, no physician)
36     if rng.random() < LAB_IMAGING_PROB[pat.esi]:
37         if rng.random() < 0.5:
38             yield env.timeout(rng.lognormal(*LAB_PARAMS))
39             pat.lab_end = env.now
40         else:
41             yield env.timeout(rng.lognormal(*IMAGING_PARAMS))
42             pat.imaging_end = env.now
43
44     # 6. Disposition
45     if rng.random() < ADMISSION_PROB[pat.esi]:
46         pat.disposition = "admitted"

```

```
47         pat.boarding_start = env.now
48         yield env.timeout(rng.exponential(self.boarding_mean_minutes))
49         pat.boarding_end = env.now
50     else:
51         pat.disposition = "discharged"
52
53     pat.depart_time = env.now
54     # room RELEASED here
```

B.3 Reproducibility

Python: 3.11 SimPy: 4.1.1 NumPy: ≥ 2.0 Base seed: 42
Replication seeds: `np.random.SeedSequence(42).spawn(50)` Warmup: 2,880 min · Horizon: 43,200 min
`python showcase/ep01/experiments.py` # ~65 s; writes results/*.csv
`python showcase/ep01/analysis.py` # writes figures/*.pdf
`cd showcase/ep01/report`
`lualatex ep01_report.tex && lualatex ep01_report.tex`